

Dukkan OS — AI Coder Prompt Pack

Offline-first business app for Algerian micro-merchants

Stack: Expo SDK 57 · Expo Router · Expo SQLite · TypeScript strict mode · i18next · EAS Build

Languages: Arabic (ar) · French (fr) · English (en) — French is default on first launch

Currency: DZD — stored as integer centimes internally

How to Use This Document

1. **Prompt 0 (Global Project Rules)** must be pasted at the top of every single task you give your AI coder. It establishes the non-negotiable rules that apply across all tasks.
2. **Prompts 1–27** are sequential. Complete each one and verify its acceptance criteria before starting the next.
3. Each prompt is self-contained — it includes the role, context, tech stack, file targets, rules, and acceptance criteria.
4. After each task, the AI coder must report: files created/modified, dependencies added, commands run, tests added/changed, manual test steps, and known limitations.
5. Run `npx tsc --noEmit` and relevant tests after every task.

Prompt 0 — Global Project Rules

Paste this at the top of every task prompt below.

You are the lead Expo and React Native engineer for Dukkan OS.

Dukkan OS is an offline-first business app for Algerian micro-businesses:

- Neighborhood grocery shops
- Home bakers
- Instagram sellers
- Market vendors
- Small service sellers

The app helps merchants:

- Record sales quickly
- Track customer credit or tabs
- Manage simple inventory
- Receive low-stock alerts
- Calculate daily estimated profit
- Share a small product catalogue
- Work without internet access

TECHNOLOGY:

- Expo SDK 57
- Use the React Native version selected by Expo SDK 57
- TypeScript strict mode
- Expo Router (file-based routing, src/app directory)
- Expo SQLite
- expo-localization
- i18next + react-i18next
- Expo development builds (not Expo Go for native features)
- EAS Build for preview and production builds
- Use npx expo install for Expo packages
- Run npx expo-doctor after dependency changes
- Never manually override Expo's React Native version
- Do not use unsupported canary packages
- Do not add a native dependency without checking Expo SDK 57 compatibility

ROUTE ARCHITECTURE:

- The only application route folder is src/app
- Every file inside src/app is a route or route layout
- Do not place components, hooks, services, repositories, types, utilities, constants, stores, or tests inside src/app
- Non-route code must live in sibling directories under src
- Route files must be thin wrappers that import feature screens
- Expo Router auto-detects src/app – no manual configuration needed

SUPPORTED LANGUAGES:

- Arabic: ar
- French: fr
- English: en
- Do not implement Darija in the MVP
- French is the default language on first launch

CRITICAL LAYOUT RULE:

- The entire application must remain visually LTR regardless of selected language
- Arabic, French, and English must use the same left-to-right screen architecture
- Do not use `I18nManager.forceRTL(true)`
- Do not use `I18nManager.allowRTL(true)`
- Do not enable native RTL layout
- Do not reverse tab order, card order, form order, navigation order, or button order for Arabic
- Do not automatically mirror the entire interface
- Keep back arrows, forward arrows, tab order, and directional controls visually LTR in all languages
- Arabic text may use right alignment inside an individual text component or input when appropriate, but the surrounding layout remains LTR
- Use explicit `writingDirection` and `textAlign` only at the text level
- Do not use global row-reverse behavior
- Do not change application direction when language changes
- Language switching must never trigger an RTL reload

MONEY SAFETY:

- Store money as integer centimes
- Never use floating-point arithmetic for money
- Use SQLite transactions for sales, payments, returns, cancellations, and inventory changes
- Never physically delete financial records
- Store historical cost on each sale item

- Public catalogues must never expose customer data, debt, cost prices, profit, or internal stock quantities
- Destructive actions require confirmation

BUSINESS RULES:

- Today means the merchant's local calendar day
- Sales may be cash, electronic, mixed, partially paid, or credit
- Customer debt equals valid unpaid sale balances minus recorded payments
- Estimated profit equals sale revenue minus historical cost of goods sold
- Stock decreases only after sale confirmation
- Cancelled and returned sales restore stock correctly
- Cancelled and returned sales do not count toward normal revenue, profit, or customer debt
- Every financial change must remain traceable

VISUAL DESIGN:

- Practical and fast
- Mobile-first
- Optimized for compact Android devices
- Safe-area aware
- High contrast
- Minimum 16px body text
- Large touch targets
- Off-white background
- Dark charcoal text
- Deep green for positive status and money
- Amber for warnings
- Red only for destructive actions and errors
- Avoid complex corporate accounting visuals

CODING STANDARDS:

- Do not add new dependencies unless necessary
- If adding a dependency, justify it and use `npx expo install` for Expo packages
- Do not implement features outside the current task
- Do not move route files except where explicitly asked
- Keep route files thin wrappers
- All user-facing strings must come from translation dictionaries
- Never hard-code visible strings inside components
- Screens must never execute raw SQL
- Use typed translation dictionaries with identical keys across all languages
- User-entered content must not be automatically translated

AFTER EVERY TASK, REPORT:

1. Files created or modified
2. Dependencies added
3. Commands run
4. Tests added or changed
5. Manual test steps
6. Known limitations

Phase 1 — Project Setup

Prompt 1 — Scaffold Expo SDK 57 Project

[GLOBAL PROJECT RULES]

TASK: Create a new Dukkan OS application using Expo SDK 57.

Run this command:

```
npx create-expo-app@latest DukkanOS --template default@sdk-57
```

Then configure the project:

1. Ensure the route directory is at src/app (move if the template creates a root app/ directory)
2. Delete the root app/ directory if it exists – src/app takes precedence
3. Remove any legacy EXPO_ROUTER_APP_ROOT env variables from babel.config.js if present
4. Remove babel-plugin-module-resolver from babel.config.js if present
5. Clear Metro cache: `rm -rf .expo node_modules/.cache`

Configure TypeScript strict mode.

Use these dependencies:

- Expo Router
- Expo SQLite
- expo-localization
- expo-dev-client
- expo-file-system
- expo-sharing
- expo-print (for PDF generation)
- i18next
- react-i18next
- AsyncStorage (for simple preferences if needed)

Do not manually install or override a separate React Native version.

ACCEPTANCE CRITERIA:

- The project starts successfully with `npx expo start -c`
- Expo Router detects routes inside src/app
- No root app/ directory exists
- No legacy EXPO_ROUTER_APP_ROOT or babel-plugin-module-resolver hacks remain
- TypeScript strict mode is enabled
- `npx expo-doctor` reports no issues
- The app runs in an Expo development build (not Expo Go)

Prompt 2 — TypeScript Path Aliases & Architecture Enforcement

[GLOBAL PROJECT RULES]

TASK: Configure TypeScript path aliases and establish the project directory structure.

Create the following directory structure under src/:

```
src/
  app/                # Routes and layouts ONLY
  components/
    ui/
    navigation/
    products/
    sales/
    customers/
    dashboard/
    catalogue/
  features/
    onboarding/
    business/
    products/
    inventory/
    sales/
    customers/
    dashboard/
    catalogue/
    settings/
    voice/
  database/
    repositories/
  localization/
  providers/
  hooks/
  services/
    sales/
    inventory/
    customers/
    catalogue/
    voice/
    export/
  stores/
  types/
  utils/
  constants/
```

Update tsconfig.json:

```
{
  "extends": "expo/tsconfig.base",
  "compilerOptions": {
    "strict": true,
    "baseUrl": ".",
    "paths": {
      "@/*": ["/src/*"],
      "@app/*": ["/src/app/*"],
      "@components/*": ["/src/components/*"],
      "@features/*": ["/src/features/*"],
      "@database/*": ["/src/database/*"],
      "@localization/*": ["/src/localization/*"],
      "@providers/*": ["/src/providers/*"],
      "@hooks/*": ["/src/hooks/*"],
      "@services/*": ["/src/services/*"],
```

```

    "@stores/*": ["/src/stores/*"],
    "@types/*": ["/src/types/*"],
    "@utils/*": ["/src/utils/*"],
    "@constants/*": ["/src/constants/*"]
  }
},
"include": ["**/*.ts", "**/*.tsx", ".expo/types/**/*.ts", "expo-env.d.ts"]
}

```

Verify package.json main is "expo-router/entry".

Create a check script at scripts/check-architecture.ts that:

- Scans src/app/ for any file that is NOT a route or layout
- Fails if it finds components, hooks, services, repositories, types, utilities, constants, stores, or tests inside src/app/
- Reports violations with file paths

ACCEPTANCE CRITERIA:

- npx tsc --noEmit passes with path aliases resolving correctly
- All sibling directories exist under src/
- package.json main is "expo-router/entry"
- The architecture check script runs and reports no violations
- Importing from @/components/... works in a test file

Phase 2 — Foundation

Prompt 3 — Theme & Design Tokens

[GLOBAL PROJECT RULES]

TASK: Create the theme system and design tokens for Dukkan OS.

Create:

- src/constants/theme.ts – color palette, spacing, typography, border radius, shadows

Color palette:

- Background: off-white (#F8F7F4 or similar)
- Text: dark charcoal (#1A1A1A or similar)
- Primary / positive / money: deep green (#1B6B3A or similar)
- Warning: amber (#D97706 or similar)
- Destructive / error: red (#B91C1C or similar)
- Muted text: gray (#6B7280 or similar)
- Surface: white (FFFFFF)
- Border: light gray (#E5E5E5 or similar)

Typography:

- Minimum 16px body text
- Large headings
- Support for Arabic, French, and English text rendering

Spacing scale: 4, 8, 12, 16, 20, 24, 32, 40, 48

Create:

- src/utils/money.ts – format centimes to DZD display string using Intl.NumberFormat with locale (ar-DZ, fr-DZ, en-DZ)
- src/utils/dates.ts – format dates using Intl.DateTimeFormat with locale
- src/utils/text.ts – text utilities including Arabic text alignment helper

ACCEPTANCE CRITERIA:

- Theme constants are exported and typed
- money.ts converts 14000 centimes to "140 DZD" or "١٤٠ دج" based on locale
- dates.ts formats dates in all three locales
- npx tsc --noEmit passes

Prompt 4 – Primitive UI Components

[GLOBAL PROJECT RULES]

TASK: Create the primitive UI component library.

Create these components under src/components/ui/:

1. AppText.tsx
 - Accepts an optional align prop: left | right | center | auto
 - For Arabic body content, use right alignment only when visually appropriate
 - Do not cause parent layout direction to change
 - Uses theme typography
2. AppScreen.tsx
 - Safe-area aware container
 - Accepts children and optional header
 - Off-white background
3. AppHeader.tsx
 - Title, optional back button, optional right action
 - Back arrow always points in fixed LTR direction
 - Do not mirror for Arabic
4. PrimaryButton.tsx
 - Deep green background
 - Large touch target (min 48px height)
 - Disabled state
 - Loading state
5. SecondaryButton.tsx
 - Outlined style
 - Same touch target rules
6. IconButton.tsx
 - Accessible labels for icon-only buttons
 - Large touch target
7. Card.tsx
 - Surface background, border, padding
 - Consistent shadow
8. EmptyState.tsx

- Icon, title, optional subtitle
 - All text translated
9. LoadingState.tsx
- Activity indicator with translated message
10. ErrorState.tsx
- Error message, retry button
 - All text translated
11. ConfirmDialog.tsx
- Title, message, confirm button, cancel button
 - Destructive confirm uses red
 - All text translated
12. MoneyText.tsx
- Displays formatted DZD amount from centimes
 - Uses @/utils/money
 - Green for positive, red for negative
13. SearchInput.tsx
- Text input with search icon
 - Translated placeholder
14. FormField.tsx
- Label, input, error message
 - Proper input labels for accessibility
 - Translated labels and errors
15. StatusBadge.tsx
- Color-coded status indicator
 - Green for paid, amber for partial, red for cancelled

All components must:

- Use @/components/ui imports internally
- Use theme tokens from @/constants/theme
- Never hard-code user-facing strings
- Support Arabic text right-alignment at the text level only
- Never reverse layout direction

ACCEPTANCE CRITERIA:

- All 15 components exist and are typed
- npx tsc --noEmit passes
- No user-facing strings are hard-coded
- Components render correctly in a test screen

Prompt 5 — Navigation, Layout & Providers

[GLOBAL PROJECT RULES]

TASK: Create the root layout, tab navigation, and provider structure.

Create route files ONLY under src/app/:

src/app/

```

_layout.tsx      # Root layout – wraps app in providers
index.tsx        # Entry – redirects to onboarding or tabs
onboarding.tsx   # Placeholder route (implemented in Phase 5)
(tabs)/
  _layout.tsx    # Tab bar layout
  index.tsx      # Home / dashboard (placeholder)
  sell.tsx       # Sell tab (placeholder)
  products.tsx   # Products tab (placeholder)
  customers.tsx  # Customers tab (placeholder)
  more.tsx       # More / settings (placeholder)
+not-found.tsx  # 404 route

```

Create providers under `src/providers/`:

1. `AppProviders.tsx`
 - Wraps the entire app
 - Composes `LocaleProvider` and `DatabaseProvider`
 - Shows loading gate until localization and database are initialized
2. `LocaleProvider.tsx`
 - Initializes `i18next`
 - Detects device locale with `expo-localization`
 - Loads persisted locale preference
 - Default to French if unsupported
 - Provides current locale and `changeLocale` function
 - Does NOT call `I18nManager.forceRTL` or `allowRTL`
3. `DatabaseProvider.tsx`
 - Initializes SQLite database
 - Runs migrations
 - Provides database instance to children
 - Shows loading state during initialization

Create tab bar component:

`src/components/navigation/AppTabBar.tsx`

Tab configuration (5 tabs, always LTR order):

1. Home – dashboard icon
2. Sell – cart icon
3. Products – box icon
4. Customers – people icon
5. More – menu icon

Tab rules:

- Display translated labels
- Keep tab order LTR in Arabic, French, and English
- Do not mirror tab bar for Arabic
- Keep all tab icons in the same order for every language

Create error boundary behavior in `_layout.tsx`:

- Catch render errors
- Show `ErrorState` with `retry`

Create loading gate:

- Show `LoadingState` until providers are ready
- Do not render main app until localization and database are initialized

ACCEPTANCE CRITERIA:

- Fresh app opens to onboarding placeholder
- Tab bar renders with 5 tabs in correct LTR order
- Switching language (when implemented) does not reverse tab order
- Error boundary catches and displays ErrorState
- Loading gate shows during provider initialization
- No files other than route files exist in src/app/
- npx tsc --noEmit passes
- npx expo start -c boots the app

Prompt 6 — Loading, Not-Found & Error States

[GLOBAL PROJECT RULES]

TASK: Finalize the foundation error handling and not-found behavior.

Ensure src/app/+not-found.tsx:

- Displays a translated "page not found" message
- Provides a button to return to Home

Create a reusable error boundary component:

src/components/ui/ErrorBoundary.tsx

- Class component that catches errors
- Shows ErrorState with retry
- Logs error details (console.error for now)

Ensure the root _layout.tsx wraps all content in ErrorBoundary.

Create src/utils/testing.ts with helper functions:

- renderWithProviders – wraps a component in AppProviders for testing
- mockDatabase – returns a mock database instance
- mockLocale – sets up i18next with a test locale

ACCEPTANCE CRITERIA:

- +not-found route renders translated message
- ErrorBoundary catches errors and shows retry
- Testing utilities are exported and typed
- npx tsc --noEmit passes
- No non-route files exist in src/app/

Phase 3 — Localization

Prompt 7 — i18n Core System

[GLOBAL PROJECT RULES]

TASK: Implement the complete localization system using i18next and react-i18next.

Create:

src/localization/

```
types.ts
i18n.ts
localeConfig.ts
```

Define types:

```
type Locale = "ar" | "fr" | "en"
type AppLayoutDirection = "ltr"
type TextAlignment = "left" | "right" | "center"
```

Localization rules:

- All three dictionaries must use identical translation keys
- Missing keys must fail tests
- Create typed `t()` helper
- Support interpolation for names, amounts, quantities, counts, and dates
- Never concatenate translated sentence fragments
- Use complete sentence translations
- Never hard-code user-facing strings in screens or components

Device locale:

- Detect device locale with `expo-localization`
- Suggest Arabic, French, or English during onboarding
- Default to French if unsupported
- Merchant selection overrides device locale
- Persist selected locale in `AsyncStorage`
- Load locale before rendering main app

Formatting:

- Arabic: `ar-DZ`
- French: `fr-DZ`
- English: `en-DZ`
- Currency: `DZD`
- Use `Intl.NumberFormat`
- Use `Intl.DateTimeFormat`

Create a typed `useTranslation` hook:

```
src/hooks/useTranslation.ts
```

- Wraps `react-i18next`'s `useTranslation`
- Returns typed `t` function, current locale, and `changeLocale` function

Create `useLocale` hook:

```
src/hooks/useLocale.ts
```

- Returns current locale
- Returns `isRTL` (always `false`)
- Returns `changeLocale` function
- Does NOT call `I18nManager` APIs

Update `LocaleProvider` to use this `i18n` configuration.

ACCEPTANCE CRITERIA:

- `i18next` initializes with French as default
- `t()` function is typed and returns translated strings
- Interpolation works for names, amounts, and dates
- Changing locale does not reload the app
- Changing locale does not trigger RTL
- `npx tsc --noEmit` passes

Prompt 8 — Translation Dictionaries & Tests

[GLOBAL PROJECT RULES]

TASK: Create complete translation dictionaries for Arabic, French, and English.

Create:

```
src/localization/  
  ar.ts  
  fr.ts  
  en.ts  
  translations.test.ts
```

Required translation sections (all must exist in all three languages):

1. common – general UI labels, buttons, actions
2. onboarding – onboarding flow strings
3. navigation – tab labels, headers
4. dashboard – dashboard cards, summaries
5. products – product management
6. inventory – stock, movements
7. sales – sale flow, cart, checkout
8. customers – customer management
9. payments – payment recording
10. catalogue – catalogue generation
11. receipts – receipt text
12. exports – CSV export labels
13. settings – settings sections
14. voice – voice command messages
15. errors – error messages
16. confirmations – confirmation dialog messages
17. emptyStates – empty list messages
18. permissions – permission request messages

Rules:

- All three dictionaries must use identical keys
- Never hard-code visible strings in components
- User-entered content (product names, customer names, notes, phone numbers, SKUs) must not be automatically translated
- Arabic strings may use `textAlign: "right"` when appropriate
- Arabic multiline text may use `writingDirection: "rtl"` at the text level only

Create `translations.test.ts`:

- Verify all three dictionaries have identical keys
- Verify no keys are missing in any language
- Verify interpolation placeholders match across languages
- Fail on any missing key

ACCEPTANCE CRITERIA:

- Arabic, French, and English dictionaries are complete
- All 18 sections exist in all three languages
- Translation key equality test passes
- No visible strings are hard-coded in components
- `t()` returns correct strings for all keys in all languages

- npx tsc --noEmit passes
- npm test passes

Phase 4 — Database

Prompt 9 — Database Schema & Migrations

[GLOBAL PROJECT RULES]

TASK: Implement the SQLite database schema, migrations, and initialization.

Use:

- expo-sqlite (current Expo SDK 57 SQLite APIs)
- Prepared statements
- Transactions
- Foreign keys
- WAL mode where supported

Create:

```
src/database/  
  database.ts      # Database initialization and connection  
  migrations.ts    # Versioned migration runner  
  schema.ts        # Table definitions as SQL strings  
  seed.ts          # Development-only seed data
```

Create a migration-version table to track applied migrations.

Tables:

business_profiles:

```
id, business_name, owner_name, business_type, currency,  
selected_locale, created_at, updated_at
```

products:

```
id, name, sku (nullable), category (nullable),  
sale_price_centimes, cost_price_centimes,  
stock_quantity, minimum_stock_quantity, unit,  
is_active, created_at, updated_at
```

customers:

```
id, name, phone (nullable), note (nullable),  
is_active, created_at, updated_at
```

sales:

```
id, customer_id (nullable), status,  
subtotal_centimes, discount_centimes, total_centimes,  
amount_paid_centimes, remaining_balance_centimes,  
payment_method, note (nullable), sold_at, created_at, updated_at
```

sale_items:

```
id, sale_id, product_id, product_name_snapshot,  
quantity, unit_sale_price_centimes, unit_cost_price_centimes,  
line_total_centimes, created_at
```

```

customer_payments:
  id, customer_id, amount_centimes, payment_method,
  note (nullable), paid_at, created_at

inventory_movements:
  id, product_id, movement_type, quantity_change,
  reference_sale_id (nullable), note (nullable), created_at

Rules:
- Money is integer centimes – no floating-point money columns
- Migrations run once and are versioned
- Seed data is development-only
- Database.ts provides a singleton database instance
- Enable foreign keys on every connection
- Enable WAL mode where supported

Create types:
src/types/
  entities.ts    # BusinessProfile, Product, Customer, Sale, SaleItem,
                  # CustomerPayment, InventoryMovement
  common.ts      # Result, PaginatedResult, ID types

ACCEPTANCE CRITERIA:
- Database initializes on app start
- Migrations run once and are tracked
- All tables are created with correct columns and constraints
- Foreign keys are enforced
- WAL mode is enabled where supported
- Seed data only loads in development
- Data persists after app restart
- npx tsc --noEmit passes

```

Prompt 10 — Product & Customer Repositories

```

[GLOBAL PROJECT RULES]

TASK: Create repositories for products and customers.

Create:
src/database/repositories/
  businessProfileRepository.ts
  productRepository.ts
  customerRepository.ts

businessProfileRepository:
- get(): BusinessProfile | null
- create(profile): BusinessProfile
- update(profile): BusinessProfile

productRepository:
- getAll(filters?): Product[]
- getById(id): Product | null
- search(query, filters?): Product[]
- create(product): Product

```

```
- update(id, product): Product
- archive(id): void (sets is_active = false, does not delete)
- adjustStock(id, quantityChange, reason): InventoryMovement
- getInventoryHistory(id): InventoryMovement[]
```

customerRepository:

```
- getAll(filters?): Customer[]
- getById(id): Customer | null
- search(query): Customer[]
- create(customer): Customer
- update(id, customer): Customer
- archive(id): void (sets is_active = false, does not delete)
```

Rules:

- Use prepared statements
- Screens must never execute raw SQL
- Archive instead of delete
- Archived products cannot be sold
- Archived customers retain history
- Stock adjustments create inventory_movement records
- Opening stock creates stock_addition movement
- Manual stock changes create stock_adjustment movement
- Require adjustment reason for manual stock changes

Add tests:

```
src/database/repositories/__tests__/
  productRepository.test.ts
  customerRepository.test.ts
  businessProfileRepository.test.ts
```

Test cases:

- Create product, retrieve by id
- Archive product, verify it cannot be sold
- Search by name and SKU
- Stock adjustment creates movement
- Create customer, retrieve by id
- Archive customer, verify history is retained

ACCEPTANCE CRITERIA:

- All repository methods work with prepared statements
- Archive sets is_active = false without deleting
- Stock adjustments create inventory movements
- Tests pass
- npx tsc --noEmit passes

Prompt 11 — Sales, Payment & Inventory Repositories (High-Risk)

[GLOBAL PROJECT RULES]

TASK: Create repositories for sales, payments, and inventory movements.
This is the highest-risk data-integrity area – all writes must be atomic.

Create:

```
src/database/repositories/
  saleRepository.ts
```

```
inventoryRepository.ts
dashboardRepository.ts
exportRepository.ts
```

saleRepository:

- create(saleInput): Sale – creates sale, sale_items, stock deductions, and inventory movements in a SINGLE transaction
- getById(id): Sale (with sale items)
- getAll(filters?): Sale[]
- search(query, filters?): Sale[]
- cancel(id, reason): void – atomic: restore stock, create movements, set status to cancelled, do NOT delete
- return(id, reason): void – atomic: restore stock, create movements, set status to returned, do NOT delete
- getByCustomerId(customerId): Sale[]
- getTodaySales(): Sale[]
- getSalesByDateRange(start, end): Sale[]

inventoryRepository:

- getMovements(productId): InventoryMovement[]
- getLowStockProducts(): Product[]
- getOutOfStockProducts(): Product[]

dashboardRepository:

- getTodayRevenue(): number (centimes)
- getTodayCost(): number (centimes)
- getTodayProfit(): number (centimes)
- getOutstandingDebt(): number (centimes)
- getLowStockCount(): number
- getRecentSales(limit): Sale[]
- getLowStockProducts(limit): Product[]
- getSevenDaySales(): { date, total }[]

exportRepository:

- getAllProducts(): Product[]
- getAllCustomers(): Customer[]
- getAllSales(): Sale[]
- getAllSaleItems(): SaleItem[]
- getAllPayments(): CustomerPayment[]
- getAllInventoryMovements(): InventoryMovement[]

CRITICAL TRANSACTION RULES:

- Sale creation is atomic: sale record + sale items + stock deductions + inventory movements – ALL or NOTHING
- Cancellation is atomic: set status + restore stock + create movements
- Return is atomic: set status + restore stock + create movements
- Historical sale items store product_name_snapshot and unit_cost_price_centimes (historical cost is preserved even if product cost changes later)
- Cancelled and returned sales are never physically deleted
- Cancelled and returned sales do not count toward revenue, profit, or debt
- Customer debt = valid unpaid sale balances - recorded payments
- Cancelled and returned sales are excluded from debt calculation

Add tests:

```
src/database/repositories/__tests__/  
  saleRepository.test.ts
```

```
dashboardRepository.test.ts
```

Test cases:

- Create sale with 2 items → verify stock decreased, movements created
- Cancel sale → verify stock restored, status is cancelled, record not deleted
- Return sale → verify stock restored, status is returned
- Cancelled sale excluded from revenue
- Returned sale excluded from debt
- Partial sale stores paid and remaining amounts
- Transaction rollback on failure (simulate error mid-transaction)

ACCEPTANCE CRITERIA:

- Sale creation is fully atomic
- Cancellation restores stock atomically
- Return restores stock atomically
- Historical costs are preserved on sale items
- Cancelled and returned sales are excluded from revenue/profit/debt
- Tests pass
- `npx tsc --noEmit` passes

Phase 5 — Onboarding

Prompt 12 — Onboarding Flow

[GLOBAL PROJECT RULES]

TASK: Implement the onboarding flow.

Route file (thin wrapper):

```
src/app/onboarding.tsx — imports OnboardingScreen from @features/onboarding
```

Create:

```
src/features/onboarding/  
  OnboardingScreen.tsx  
  components/  
    BusinessNameStep.tsx  
    OwnerNameStep.tsx  
    BusinessTypeStep.tsx  
    LanguageStep.tsx  
src/hooks/useOnboarding.ts
```

Onboarding fields:

- Business/shop name (required)
- Merchant name (required)
- Business type (selectable: grocery, bakery, instagram seller, market vendor, service seller, other)
- Currency: DZD (fixed, displayed but not changeable)
- Language selection: العربية | Français | English

Onboarding behavior:

- French is the default language
- Save selected language locally (AsyncStorage)
- Save business profile locally (SQLite)

- Save onboarding completion flag locally (AsyncStorage)
- Arabic language selection must NOT enable RTL
- Changing language during onboarding does not reload the app for layout direction
- Arabic text can be right-aligned where appropriate, but the onboarding screen structure remains LTR

Navigation behavior:

- Fresh app opens on onboarding
- Completing onboarding navigates to tabs
- Restarting app with completed onboarding skips to tabs

Create src/hooks/useOnboarding.ts:

- isOnboardingComplete: boolean
- completeOnboarding(profile): void
- HasOnboarding flag persisted in AsyncStorage

Update src/app/index.tsx:

- Check onboarding completion
- If complete → redirect to /(tabs)/
- If not complete → redirect to /onboarding

ACCEPTANCE CRITERIA:

- Fresh app opens on onboarding
- Completing onboarding opens tabs
- Restarting app skips onboarding
- Arabic, French, and English labels appear
- Architecture remains LTR in all three languages
- No files other than route files exist in src/app/
- npx tsc --noEmit passes
- npx expo start -c boots correctly

Phase 6 — Products

Prompt 13 — Product List, Search & Filters

[GLOBAL PROJECT RULES]

TASK: Implement the Products tab – list, search, and filters.

Route file (thin wrapper):

src/app/(tabs)/products.tsx – imports ProductListScreen from @features/products

Create:

```
src/features/products/
  ProductListScreen.tsx
  components/
    ProductCard.tsx
    ProductSearchBar.tsx
    ProductFilterTabs.tsx
src/hooks/useProducts.ts
src/components/products/
  ProductRow.tsx
```

Product list features:

- Display all active products
- Show name, SKU, sale price (formatted DZD), stock quantity, unit
- Low-stock badge when stock <= minimum_stock_quantity
- Out-of-stock badge when stock === 0

Search:

- Search by name or SKU
- Real-time filtering

Filters:

- All
- Low stock (stock <= minimum)
- Out of stock (stock === 0)
- Archived (is_active === false)

Rules:

- Validate all inputs
- Convert DZD display to integer centimes on save
- All labels, errors, and empty states are translated
- Arabic text may be right-aligned, but product screens remain LTR
- Do not mirror product rows or controls for Arabic

Add tests for useProducts hook.

ACCEPTANCE CRITERIA:

- Product list displays all products
- Search filters by name and SKU
- Filter tabs work correctly
- Low-stock and out-of-stock badges appear
- Empty states show translated messages
- All text is translated in all three languages
- npx tsc --noEmit passes

Prompt 14 — Product Add, Edit, Detail & Stock Adjustment

[GLOBAL PROJECT RULES]

TASK: Implement product creation, editing, detail view, and stock adjustment.

Route files (thin wrappers):

```
src/app/products/new.tsx — imports ProductFormScreen
src/app/products/[id].tsx — imports ProductDetailScreen
src/app/products/edit/[id].tsx — imports ProductFormScreen
```

Create:

```
src/features/products/
  ProductFormScreen.tsx
  ProductDetailScreen.tsx
components/
  ProductForm.tsx
  StockAdjustmentForm.tsx
  InventoryHistoryList.tsx
```

Product form fields:

- Name (required)
- SKU (optional)
- Category (optional)
- Sale price (required, DZD → centimes)
- Cost price (required, DZD → centimes)
- Current stock (required on create, creates stock_addition movement)
- Minimum stock threshold (required)
- Unit: Piece | Kilogram | Litre | Pack | Box | Other

Stock adjustment:

- Manual stock changes create stock_adjustment movement
- Require adjustment reason
- Stock cannot become negative unless enabled in Settings
- Opening stock creates stock_addition movement

Product detail:

- Show all product fields
- Show inventory history (movements list)
- Stock adjustment button
- Edit button
- Archive button (with confirmation)

Rules:

- Validate all fields
- Convert DZD to integer centimes
- Archive instead of delete
- Archived products cannot be sold
- Confirmation required before archive
- All text translated
- Arabic text may be right-aligned, but screens remain LTR

Add tests for product form validation and stock adjustment.

ACCEPTANCE CRITERIA:

- Create product: Lait 1L, sale price 140 DZD, cost price 110 DZD, stock 24, minimum stock 5
- Product appears immediately in list
- Product detail shows inventory history
- Product persists after restart
- Stock adjustment creates movement
- Archive requires confirmation
- Arabic, French, and English use the same screen structure
- `npx tsc --noEmit` passes

Phase 7 — Sell & Cart

Prompt 15 — Cart & Checkout Flow

[GLOBAL PROJECT RULES]

TASK: Implement the Sell tab with cart and checkout.

Route file (thin wrapper):

src/app/(tabs)/sell.tsx — imports SellScreen from @features/sales

Create:

```
src/features/sales/  
  SellScreen.tsx  
  components/  
    ProductSearchSheet.tsx  
    CartList.tsx  
    CartItem.tsx  
    CartSummary.tsx  
    CheckoutSheet.tsx  
    ReceiptPreview.tsx  
src/stores/cartStore.ts  
src/hooks/useSales.ts
```

Cart features:

- Product search (by name or SKU)
- Recently sold products (quick add)
- Cart with large plus/minus controls
- Remove item
- Subtotal display
- Optional discount
- Total display
- Clear cart (with confirmation if non-empty)

Checkout:

- Payment method selection: Cash | Electronic | Credit | Partial payment
- Credit and partial payment require customer selection
- Default payment is cash
- Amount paid input for partial payments
- Sale success state
- Receipt preview

Cart store (cartStore.ts):

- items: CartItem[]
- addItem(productId, quantity)
- removeItem(productId)
- updateQuantity(productId, quantity)
- clearCart()
- preserveCart flag (persist during temporary navigation)
- Clear cart after successful sale only

Rules:

- Empty cart cannot be confirmed
- Insufficient stock blocks sale unless allow-negative-stock is enabled
- Credit and partial payment require customer
- Sale confirmation creates sale, items, stock deductions, and inventory movements atomically
- Preserve cart during temporary navigation

- Clear cart after successful sale only
- Confirm before discarding non-empty cart
- Use selected language for all labels and messages
- Keep the exact same LTR visual structure in all languages
- Arabic text may align right within labels or fields, but cart, buttons, controls, and navigation do not reverse

Add tests for cart store logic.

ACCEPTANCE CRITERIA:

- Sell two Lait 1L products → total is 280 DZD
- Stock decreases only after confirmation
- Credit sale requires customer selection
- Partial payment stores paid and remaining amounts
- Cart is preserved during navigation
- Cart clears after successful sale
- Manual flow works fully offline
- `npx tsc --noEmit` passes

Prompt 16 — Sale Confirmation & Receipt

[GLOBAL PROJECT RULES]

TASK: Implement sale confirmation, receipt generation, and post-sale navigation.

Create:

```
src/features/sales/
  components/
    SaleConfirmation.tsx
    Receipt.tsx
src/services/sales/
  saleService.ts
  saleCalculator.ts
```

saleService.ts:

- `createSale(cartItems, paymentConfig, customerId?): Sale`
 - calls `saleRepository.create()` in a transaction
 - clears cart on success
 - returns created sale with items
- `validateSale(cartItems, paymentConfig): ValidationResult`
 - check stock availability
 - check customer required for credit/partial
 - return errors

saleCalculator.ts:

- `calculateSubtotal(items): number (centimes)`
- `calculateTotal(subtotal, discount): number (centimes)`
- `calculateChange(amountPaid, total): number (centimes)`
- `calculateProfit(saleItems): number (centimes)`
 - uses historical `cost_price_centimes` from sale items

Receipt:

- Show business name, date, sale items with quantities and prices, subtotal, discount, total, payment method, amount paid, change
- All text translated

- Receipt text can be shared via native share sheet
- Arabic receipt text may be right-aligned, but structure remains LTR

Post-sale navigation:

- After successful sale → show success state with receipt preview
- Navigate to sale detail or back to sell screen

Add tests for saleCalculator:

- Subtotal calculation
- Total with discount
- Change calculation
- Profit calculation using historical cost

ACCEPTANCE CRITERIA:

- Sale confirmation creates sale atomically
- Receipt shows all required information
- Receipt can be shared
- Profit calculation uses historical cost, not current cost
- Cart clears after successful sale
- All receipt text is translated
- `npx tsc --noEmit` passes
- Tests pass

Phase 8 — Customers

Prompt 17 — Customer List, Search & Detail

[GLOBAL PROJECT RULES]

TASK: Implement the Customers tab with list, search, and detail view.

Route file (thin wrapper):

`src/app/(tabs)/customers.tsx` — imports `CustomerListScreen`

Route files:

`src/app/customers/new.tsx` — imports `CustomerFormScreen`

`src/app/customers/[id].tsx` — imports `CustomerDetailScreen`

`src/app/customers/edit/[id].tsx` — imports `CustomerFormScreen`

Create:

`src/features/customers/`
 `CustomerListScreen.tsx`
 `CustomerFormScreen.tsx`
 `CustomerDetailScreen.tsx`
 `components/`
 `CustomerCard.tsx`
 `CustomerSearchBar.tsx`
 `CustomerFilterTabs.tsx`
`src/hooks/useCustomers.ts`
`src/components/customers/`
 `CustomerRow.tsx`

Customer list:

- Display all active customers
- Show name, phone, outstanding balance, debt status
- Search by name or phone

Filters:

- All
- With debt
- No debt
- Archived

Customer detail:

- Current debt (calculated, not a stored field)
- Record payment button
- Credit-sale history
- Payment history
- Activity timeline
- Notes
- Reminder share action

Customer form:

- Name (required)
- Phone (optional)
- Note (optional)

Rules:

- Archive instead of delete
- Archived customers retain history
- All text translated
- Arabic text may be right-aligned, but screens remain LTR

Add tests for useCustomers hook and customer listing.

ACCEPTANCE CRITERIA:

- Customer list displays all customers
- Search filters by name and phone
- Filter tabs work
- Customer detail shows debt, history, and notes
- Customer persists after restart
- `npx tsc --noEmit` passes

Prompt 18 — Customer Payments & Debt Tracking

[GLOBAL PROJECT RULES]

TASK: Implement customer payment recording, debt calculation, and reminder sharing.

Create:

```
src/features/customers/  
  components/  
    RecordPaymentSheet.tsx  
    DebtSummary.tsx  
    PaymentHistory.tsx  
src/services/customers/  
  customerBalanceService.ts
```

customerBalanceService.ts:

- getCustomerDebt(customerId): number (centimes)
 - valid unpaid sale balances minus recorded payments
 - cancelled and returned sales are excluded
- getCustomerPayments(customerId): CustomerPayment[]
- recordPayment(customerId, amount, method, note): CustomerPayment
 - atomic transaction
- canRecordPayment(customerId, amount): boolean
 - block overpayment in MVP

Payment rules:

- Cash or electronic
- Amount must be positive
- Note optional
- Confirmation required
- Block overpayment in MVP
- Record payment transactionally

Debt calculation:

- Debt = valid unpaid sale balances - customer payments
- Cancelled and returned sales do not count
- Do not use a mutable debt field as the source of truth
- Calculate debt dynamically from sales and payments

Reminder sharing:

- Generate reminder text in selected language
- Arabic: "الرصيد المتبقي لديك لدى [businessName] مرحباً [customerName]. دج. شكراً [amount] هو"
- French: "Bonjour [customerName], votre solde restant chez [businessName] est de [amount] DZD. Merci."
- English: "Hello [customerName], your remaining balance at [businessName] is [amount] DZD. Thank you."
- Use native share sheet
- Arabic reminder text may be right-aligned, but share sheet remains LTR

Add tests for customerBalanceService:

- Debt calculation with multiple sales and payments
- Cancelled sale excluded from debt
- Returned sale excluded from debt
- Overpayment blocked
- Payment recorded transactionally

ACCEPTANCE CRITERIA:

- Create Ahmed Benali
- Create 1,000 DZD credit sale to Ahmed
- Ahmed debt is 1,000 DZD
- Record payment of 400 DZD
- Debt becomes 600 DZD
- Reminder is generated in selected language
- Cancelled sale excluded from debt
- Data persists after restart
- npx tsc --noEmit passes
- Tests pass

Phase 9 — Dashboard

Prompt 19 — Home Dashboard

[GLOBAL PROJECT RULES]

TASK: Implement the Home dashboard using real SQLite data.

Route file (thin wrapper):

```
src/app/(tabs)/index.tsx — imports DashboardScreen from @features/dashboard
```

Create:

```
src/features/dashboard/  
  DashboardScreen.tsx  
  components/  
    GreetingCard.tsx  
    SummaryCards.tsx  
    SalesChart.tsx  
    RecentSalesList.tsx  
    LowStockList.tsx  
    QuickActions.tsx  
src/hooks/useDashboard.ts  
src/components/dashboard/  
  SummaryCard.tsx  
  QuickActionButton.tsx
```

Dashboard content:

- Business greeting (using business profile name)
- Current date (formatted by locale)
- Today's sales card (total revenue)
- Estimated profit card (revenue - historical cost)
- To-collect card (outstanding customer debt)
- Low-stock card (count of products at or below threshold)
- Seven-day sales chart (simple bar or line chart)
- Recent sales (newest first, limit 5)
- Low-stock products (limit 5)
- Quick actions: New sale, Add product, Add customer, Record payment

Calculations:

- Today's sales includes valid completed, partially paid, and credit sales
- Exclude cancelled and returned sales
- Estimated profit uses historical sale-item cost (not current product cost)
- To collect equals valid outstanding customer debt
- Low stock counts active products at or below threshold

Rules:

- Refresh after sales, payments, edits, cancellations, and returns
- Use translated dates, labels, and amounts
- Do not use heavy chart dependencies unnecessarily — use a simple custom bar chart component
- Arabic labels may be right-aligned, but card order and chart architecture remain LTR
- Do not mirror dashboard cards for Arabic

Add tests for useDashboard hook and dashboard calculations.

ACCEPTANCE CRITERIA:

- Revenue 280 DZD and cost 220 DZD produces 60 DZD profit
- Low-stock count updates after a sale
- Recent sales are newest first
- Dashboard works offline
- Arabic, French, and English use the same LTR layout
- Quick actions navigate to correct screens
- `npx tsc --noEmit` passes

Phase 10 — Sales History

Prompt 20 — Sales History, Cancellation & Returns

[GLOBAL PROJECT RULES]

TASK: Implement sales history, sale details, cancellation, and returns.

Route files:

`src/app/sales/history.tsx` – imports `SalesHistoryScreen`
`src/app/sales/[id].tsx` – imports `SaleDetailScreen`

Create:

`src/features/sales/`
 `SalesHistoryScreen.tsx`
 `SaleDetailScreen.tsx`
 `components/`
 `SaleCard.tsx`
 `SaleFilterTabs.tsx`
 `CancelSaleDialog.tsx`
 `ReturnSaleDialog.tsx`

Sales history filters:

- Today
- Last 7 days
- This month
- Custom range
- All
- Paid
- Partially paid
- Credit
- Cancelled
- Returned

Search:

- By customer name, product name, or sale ID

Sale detail:

- Show all sale information (items, prices, payment, status)
- Cancellation button (with reason field and confirmation)
- Return button (with reason field and confirmation)
- Receipt preview

Rules:

- Cancelled and returned sales cannot be modified again
- Require reason for cancellation and return
- Require irreversible-action confirmation
- Use database transactions
- Restore stock on cancellation and return
- Create inventory movement on cancellation and return
- Do not increase customer debt on cancellation/return
- Never delete original sale record
- Exclude cancelled and returned sales from normal revenue and profit
- Translate all statuses and messages
- Keep visual layout LTR in all languages

Add tests for cancellation and return flows.

ACCEPTANCE CRITERIA:

- Search finds sales
- Cancellation restores stock
- Return restores stock
- Revenue and profit exclude cancelled and returned sales
- Original record remains visible with status and reason
- Cancelled/returned sale cannot be modified again
- All statuses are translated
- npx tsc --noEmit passes
- Tests pass

Phase 11 — Catalogue

Prompt 21 — Public Mini-Catalogue

[GLOBAL PROJECT RULES]

TASK: Implement the public mini-catalogue feature.

Route file:

src/app/settings/catalogue.tsx — imports CatalogueScreen

Create:

```
src/features/catalogue/  
  CatalogueScreen.tsx  
  components/  
    ProductSelector.tsx  
    CataloguePreview.tsx  
    CatalogueSettings.tsx  
src/services/catalogue/  
  catalogueService.ts  
src/components/catalogue/  
  CataloguePreview.tsx
```

Use Expo-compatible modules:

- expo-file-system
- expo-sharing
- expo-print (for PDF generation)

Catalogue settings:

- Show prices toggle
- Hide out-of-stock toggle
- Contact text
- Address text
- Product selection (checkbox list)

Catalogue includes:

- Business name
- Contact
- Address
- Product names
- Sale prices (when enabled)
- Availability (Available / On order)
- Generation date

Availability translations:

- Arabic: "حسب الطلب" / "متوفر"
- French: "Disponible" / "Sur commande"
- English: "Available" / "On order"

Rules:

- Use selected language
- Never expose customers, debt, cost, profit, or internal stock quantities
- Use native share sheet
- Arabic text may be right-aligned, but catalogue document structure and application screen remain LTR
- Do not reverse the application layout
- If PDF fails, provide text-sharing fallback
- Test catalogue output in all three languages

Add tests for catalogue privacy (verify no private data in output).

ACCEPTANCE CRITERIA:

- Merchant selects products for catalogue
- Prices can be hidden
- Out-of-stock products can be hidden
- Shared output includes public data only
- Arabic, French, and English outputs are complete
- PDF generation works or text fallback works
- No customer data, debt, cost, or profit in catalogue output
- `npx tsc --noEmit` passes

Phase 12 — Voice

Prompt 22 — Voice-Assisted Sales (Optional, After Manual Flow Is Stable)

[GLOBAL PROJECT RULES]

TASK: Implement voice-assisted sales and typed quick commands.

IMPORTANT CONSTRAINTS:

- Voice must NEVER silently add products or create a sale
- Manual typed input must always remain available
- Verify the speech-recognition library supports Expo SDK 57 and development builds before installing it
- If compatibility is uncertain, implement typed quick commands only
- Do not use Arabic speech recognition unless the selected library explicitly supports it

Create:

```
src/features/voice/
  VoiceSaleInput.tsx
  components/
    VoiceButton.tsx
    ReviewSheet.tsx
    TypedQuickCommand.tsx
src/services/voice/
  voiceSaleParser.ts
  textNormalizer.ts
  productMatcher.ts
```

Features:

- Microphone button (requests permission only after button press)
- Typed quick-command field (always available as fallback)
- Text normalization (handles Arabic, French, and English number words)
- Parser (extracts quantity, product name, customer, payment method)
- Product matcher (exact match first, partial match only when one clear result)
- Review sheet (shows parsed items before cart mutation)
- Mandatory "Add to cart" confirmation

Parser rules:

- Support Arabic, French, and English number words from zero to twenty
- Match exact product names first
- Use partial matching only when one clear result exists
- Never guess in ambiguous cases – show choices
- Require customer for credit sales
- Default to cash only if payment is not specified
- Allow editing recognized text
- Require confirmation before cart mutation

Example commands:

- Arabic: "بعت حبتين حليب", "أضف ثلاث زجاجات ماء", "حبتان حليب لأحمد بالدين", "خبزة واحدة نقداً"
- French: "Vendu deux lait", "Vendu trois bouteilles d'eau", "Deux lait pour Ahmed, à crédit", "Un pain, payé cash"
- English: "Sold two milks", "Sold three water bottles", "Two milks for Ahmed, on credit", "One bread, paid in cash"

Permissions:

- Request microphone permission only after button press
- Translate permission messages into all three languages
- Preserve typed fallback if permission is denied

Layout:

- Keep app architecture LTR in all languages
- Arabic review text may be right-aligned inside the review sheet
- Do not reverse sheet structure or button order

- Do not use RTL APIs

Add tests for:

- Arabic number parsing
- French number parsing
- English number parsing
- Product ambiguity handling
- Parser edge cases

ACCEPTANCE CRITERIA:

- Typed and voice commands display review
- No cart changes before confirmation
- Ambiguous products show choices
- Arabic, French, and English command examples work
- Manual selling works when voice is unavailable
- Permission is requested only after button press
- Typed fallback works when permission is denied
- `npx tsc --noEmit` passes
- Tests pass

Phase 13 — Settings

Prompt 23 — Settings UI & Language

[GLOBAL PROJECT RULES]

TASK: Implement the More tab and settings sections.

Route file:

`src/app/(tabs)/more.tsx` — imports `MoreScreen`

Route files:

`src/app/settings/language.tsx` — imports `LanguageSettingsScreen`

`src/app/settings/business.tsx` — imports `BusinessSettingsScreen`

Create:

`src/features/settings/`
 `MoreScreen.tsx`
 `LanguageSettingsScreen.tsx`
 `BusinessSettingsScreen.tsx`
`components/`
 `SettingsSection.tsx`
 `LanguageSelector.tsx`

Settings sections (More tab):

- Business profile
- Language
- Inventory rules
- Catalogue
- Data export
- Data reset
- About

Language options:

- العربية
- Français
- English

Language rules:

- French is default
- Switching language applies immediately
- Do not change layout direction
- Do not call I18nManager
- Do not reload for RTL
- Keep app architecture LTR in all languages
- Arabic text may be right-aligned inside individual text blocks
- Do not show Darija

Business profile settings:

- Edit business name, owner name, business type
- Currency is DZD (display only, not editable)

ACCEPTANCE CRITERIA:

- More tab shows all settings sections
- Language switching works immediately
- Switching language does not reload or change layout direction
- Arabic text appears with LTR layout
- Business profile can be edited
- npx tsc --noEmit passes

Prompt 24 — Inventory Rules, Data Export & Data Reset

[GLOBAL PROJECT RULES]

TASK: Implement inventory settings, data export, and data reset.

Route files:

src/app/settings/export.tsx – imports ExportSettingsScreen

Create:

```
src/features/settings/  
  InventorySettingsScreen.tsx  
  ExportSettingsScreen.tsx  
  DataResetScreen.tsx  
  components/  
    ExportButton.tsx  
    ResetConfirmation.tsx  
src/services/export/  
  csvExportService.ts
```

Inventory settings:

- Allow negative stock (default: disabled)
- Default low-stock threshold
- Negative stock disabled by default

Data export:

- Export products (CSV)
- Export customers (CSV)

- Export sales (CSV)
- Export sale items (CSV)
- Export customer payments (CSV)
- Export inventory movements (CSV)
- CSV headers use selected language
- Use expo-file-system and expo-sharing
- Confirm before export
- Never upload automatically

csvExportService.ts:

- exportProducts(): generates CSV with product data
- exportCustomers(): generates CSV with customer data (no debt or cost)
- exportSales(): generates CSV with sale data
- exportSaleItems(): generates CSV with sale item data
- exportPayments(): generates CSV with payment data
- exportInventoryMovements(): generates CSV with movement data
- All CSV headers are translated

Data reset:

- Clear development/demo data
- Delete all local business data
- Require exact business-name confirmation (type the business name to confirm)
- Explain deletion cannot be undone
- Clear database and preferences
- Reset locale to French
- Return to onboarding

Rules:

- Data reset is a destructive action – require exact business-name confirmation
- CSV exports must never include cost prices, profit, or internal stock quantities in customer-facing exports
- All text translated
- Accessibility: accessible labels, large touch targets, high contrast

Add tests for CSV export format and data reset protection.

ACCEPTANCE CRITERIA:

- App exports accurate CSV data
- CSV headers use selected language
- Full deletion is protected by business-name confirmation
- Offline data persists
- TypeScript passes
- Tests pass
- Arabic, French, and English are complete
- Visual architecture remains LTR

Phase 14 — Quality & Audit

Prompt 25 — Accessibility & Passive Quality

[GLOBAL PROJECT RULES]

TASK: Ensure accessibility and passive quality across the entire app.

Accessibility requirements:

- Accessible labels for all icon-only buttons
- Large touch targets (minimum 48px height)
- High contrast (verify all text/background combinations meet WCAG AA)
- Proper input labels for all form fields
- Clear translated error messages
- No horizontal overflow on any screen
- Arabic text must not clip or truncate
- Arabic does not reverse the interface
- Directional icons remain fixed LTR
- DZD, dates, phone numbers, SKUs, and quantities remain readable in all languages

Review and fix:

- Audit all screens for text truncation on compact devices
- Verify Arabic text rendering on all screens
- Verify that switching between Arabic, French, and English does not cause any layout shift or RTL behavior
- Verify all ConfirmDialog instances require explicit confirmation for destructive actions
- Verify all error states have translated messages and retry actions

Create a lint check:

scripts/check-no-rtl.ts

- Search the entire codebase for:
 - I18nManager.forceRTL
 - I18nManager.allowRTL
 - row-reverse (in style objects)
 - flexDirection: 'row-reverse'
- Fail if any are found

Add test:

src/__tests__/ltr-architecture.test.ts

- Verify no RTL API usage in source files
- Verify fixed LTR behavior

ACCEPTANCE CRITERIA:

- No RTL API usage anywhere in the codebase
- All icon-only buttons have accessible labels
- All text is readable on compact Android devices
- Arabic text does not clip
- All errors are translated
- All destructive actions require confirmation
- LTR architecture test passes
- npx tsc --noEmit passes
- npm test passes

Prompt 26 — Complete Test Suite

[GLOBAL PROJECT RULES]

TASK: Complete the test suite covering all business rules and edge cases.

Ensure these tests exist and pass:

Money:

- Money conversion (DZD to centimes and back)
- Money formatting in all three locales
- No floating-point money calculations

Sales:

- Sale totals calculation (subtotal, discount, total)
- Profit calculation using historical cost
- Partial payment storage
- Change calculation

Customers:

- Customer debt calculation
- Cancelled sale excluded from debt
- Returned sale excluded from debt
- Payment recorded transactionally
- Overpayment blocked

Inventory:

- Stock movements on sale
- Stock restoration on cancellation
- Stock restoration on return
- Low-stock detection

Voice:

- Arabic number parsing (zero to twenty)
- French number parsing
- English number parsing
- Product ambiguity handling

Localization:

- Translation key equality across all three languages
- No missing keys
- Interpolation placeholder matching

Architecture:

- Fixed LTR behavior
- No RTL API usage
- No non-route files in src/app/

Security:

- Export privacy (no private financial data in CSV or catalogue)
- Data reset requires business-name confirmation
- Cancelled/returned sales cannot be modified again

Run:

- npx tsc --noEmit
- npm test -- --runInBand

- npx expo-doctor
- npx expo start --dev-client (verify app boots)

ACCEPTANCE CRITERIA:

- All tests pass
- TypeScript passes with no errors
- expo-doctor reports no issues
- App boots in development build

Prompt 27 — Production-Readiness Audit

[GLOBAL PROJECT RULES]

TASK: Perform a complete production-readiness audit of Dukkan OS.

ARCHITECTURE:

- Confirm the only route folder is src/app
- Confirm src/app contains route files and layouts only
- Confirm no components, hooks, services, repositories, types, utilities, stores, constants, or tests are inside src/app
- Confirm non-route code is organized into sibling directories under src
- Run scripts/check-architecture.ts

LOCALIZATION:

- Confirm Arabic, French, and English dictionaries have identical keys
- Confirm all visible strings are translated
- Confirm generated receipts, catalogues, reminders, CSV headers, errors, and permissions are translated
- Confirm user-entered data is not automatically translated

LTR REQUIREMENT:

- Search the entire codebase for:
 - I18nManager.forceRTL
 - I18nManager.allowRTL
 - row-reverse
 - automatic direction switching
 - language-triggered app reload
- Remove or reject all global RTL behavior
- Confirm Arabic uses the same LTR screen architecture as French and English
- Confirm Arabic text can be right-aligned only at the text level
- Confirm tab order, form order, card order, button order, navigation arrows, and screen structure remain LTR
- Confirm no Arabic layout mirroring occurs

DATA SAFETY:

- Confirm all money uses integer centimes
- Confirm all financial writes use transactions
- Confirm historical sale costs are stored on sale items
- Confirm cancellations and returns restore stock
- Confirm customer debt excludes cancelled and returned sales
- Confirm public catalogues contain no private financial data

QUALITY:

Run:

- npx tsc --noEmit

- npm test -- --runInBand
- npx expo-doctor
- npx expo start --dev-client
- Build and test on Android development build
- Build and test on iOS development build
- Test offline restart
- Test Arabic, French, and English
- Test compact screen sizes
- Test catalogue sharing
- Test CSV export
- Test all destructive confirmations

RETURN:

1. Final project tree
2. Dependency list
3. Commands run
4. Test results
5. Failed checks
6. Remaining limitations
7. Recommended version 1.1 features

ACCEPTANCE CRITERIA:

- All architecture checks pass
- All localization checks pass
- No RTL API usage anywhere
- All data safety checks pass
- All tests pass
- TypeScript passes
- expo-doctor reports no issues
- App runs in development build on Android and iOS
- Offline data persists after restart

Prompt 28 — End-to-End Verification Script

[GLOBAL PROJECT RULES]

TASK: Execute the end-to-end verification of Dukkan OS.

Follow these steps in order and report results:

1. Install a fresh Expo development build
2. Select Arabic
3. Confirm that:
 - Arabic text appears
 - Tab order remains Home, Sell, Products, Customers, More in the same LTR structure
 - Cards and forms remain in the same positions
 - No RTL reload occurs
 - Back arrows remain fixed in the chosen LTR design
4. Create:
 - Product: Lait 1L
 - Sale price: 140 DZD
 - Cost price: 110 DZD
 - Stock: 24

- Low-stock threshold: 5
5. Create customer Ahmed Benali
 6. Sell two products for cash:
 - Revenue: 280 DZD
 - Estimated profit: 60 DZD
 - Remaining stock: 22
 7. Sell three products to Ahmed on credit:
 - Credit: 420 DZD
 - Remaining stock: 19
 8. Record Ahmed payment of 200 DZD:
 - Remaining debt: 220 DZD
 9. Return the credit sale:
 - Stock: 22
 - Returned sale excluded from Ahmed's debt
 10. Switch Arabic to French:
 - Text changes immediately
 - Layout remains LTR
 - No reload
 11. Switch French to English:
 - Text changes immediately
 - Layout remains LTR
 - No reload
 12. Generate catalogue in all three languages
 13. Confirm private financial data is absent from catalogue
 14. Export CSV in all three languages
 15. Close and reopen offline
 16. Confirm all data and the selected language remain correct

REPORT:

- Pass/fail for each step
- Any issues encountered
- Screenshots or logs of failures
- Recommendations for fixes

Summary — Prompt Sequence

#	Phase	Prompt	Focus
0	—	Global Project Rules	Non-negotiable rules — paste at top of every task
1	Setup	Scaffold Expo SDK 57	Project creation, src/app, dependencies
2	Setup	Path Aliases & Architecture	tsconfig, directory structure, check script
3	Foundation	Theme & Design Tokens	Colors, spacing, typography, money/date utils
4	Foundation	Primitive UI Components	15 reusable components

#	Phase	Prompt	Focus
5	Foundation	Navigation, Layout & Providers	Root layout, tabs, providers, loading gate
6	Foundation	Loading, Not-Found & Error States	Error boundary, 404, test utilities
7	Localization	i18n Core System	i18next, typed t(), useTranslation, useLocale
8	Localization	Translation Dictionaries	ar/fr/en dictionaries, 18 sections, tests
9	Database	Schema & Migrations	7 tables, migrations, WAL, foreign keys
10	Database	Product & Customer Repos	CRUD, archive, stock movements
11	Database	Sales, Payment & Inventory Repos	Atomic transactions, cancel, return, debt
12	Onboarding	Onboarding Flow	Business profile, language, navigation
13	Products	Product List & Search	List, search, filters, badges
14	Products	Product Add/Edit/Detail	Form, detail, stock adjustment, archive
15	Sell	Cart & Checkout	Cart store, checkout, payment methods
16	Sell	Sale Confirmation & Receipt	Atomic sale, receipt, profit calc
17	Customers	List, Search & Detail	List, filters, form, detail
18	Customers	Payments & Debt	Payment recording, debt calc, reminders
19	Dashboard	Home Dashboard	Summary cards, chart, quick actions
20	Sales History	History, Cancel & Return	Filters, cancellation, returns
21	Catalogue	Public Mini-Catalogue	PDF, sharing, privacy
22	Voice	Voice-Assisted Sales	Parser, matcher, review sheet, fallback
23	Settings	Settings UI & Language	More tab, language switch, business profile
24	Settings	Export & Reset	CSV export, inventory rules, data reset
25	Quality	Accessibility & Passive Quality	A11y, LTR check, contrast, truncation
26	Quality	Complete Test Suite	All business rules, edge cases
27	Quality	Production-Readiness Audit	Architecture, LTR, data safety, builds
28	Quality	End-to-End Verification	Manual E2E test script

Key Principles

1. **Small, testable tasks** — Each prompt produces a verifiable checkpoint with acceptance criteria
2. **Security first** — Money as integer centimes, transactions for all financial writes, no private data in exports/catalogue, destructive actions require confirmation
3. **Maintainability** — Route files are thin wrappers, feature-based sibling directories, path aliases, no non-route code in src/app

4. **Clean architecture** — Separation of concerns: routes → features → services → repositories → database
5. **LTR always** — Arabic supported without RTL layout, no I18nManager RTL APIs
6. **Offline-first** — All data stored locally in SQLite, app works without internet
7. **No overbuilding** — Each prompt explicitly scopes to its task, no scope creep
8. **Test as you go** — Every implementation prompt includes tests for its area